



# AI-Assisted Threat Detection Dashboard

## Model Research Document

*Machine Learning & Deep Learning Approaches for Real-Time  
Cyber Threat Detection, Classification and Visualization*

Prepared by: Security Analytics & AI Research Team  
Version 1.0 | July 2026

# Table of Contents

|                                         |    |
|-----------------------------------------|----|
| 1. Abstract                             | 3  |
| 2. Introduction                         | 3  |
| 3. Review of Existing Approaches        | 4  |
| 4. Proposed System Architecture         | 4  |
| 5. Data Collection and Preprocessing    | 5  |
| 6. Model Research and Methodology       | 6  |
| 7. Feature Engineering                  | 7  |
| 8. Evaluation Methodology               | 8  |
| 9. Dashboard Design and User Experience | 8  |
| 10. Proposed Technology Stack           | 9  |
| 11. Implementation Roadmap              | 9  |
| 12. Challenges and Limitations          | 10 |
| 13. Future Research Directions          | 10 |
| 14. Conclusion                          | 10 |
| 15. References and Further Reading      | 11 |

# 1. Abstract

---

Modern enterprise networks generate a continuous, high-volume stream of security telemetry — network flows, authentication logs, endpoint events, and application traces — that vastly exceeds the capacity of manual triage. This document presents the model research and design rationale behind an **AI-Assisted Threat Detection Dashboard**, a system that combines supervised, unsupervised, and deep-learning models with an interactive visualization layer to help Security Operations Center (SOC) analysts detect, prioritize, and investigate cyber threats faster and with fewer false positives.

The research evaluates candidate algorithms across three detection paradigms — signature-adjacent supervised classification, unsupervised anomaly detection, and sequence-based deep learning — and proposes a hybrid ensemble architecture that balances detection accuracy, interpretability, latency, and operational cost. The document further covers the data pipeline, feature engineering strategy, evaluation methodology, dashboard UX considerations, and a phased implementation roadmap.

## 2. Introduction

---

### 2.1 Background

Cybersecurity teams face an asymmetry problem: attackers need to succeed once, while defenders must catch every attempt. Traditional rule-based and signature-based detection systems (SIEM correlation rules, IDS/IPS signatures) are effective against known attack patterns but struggle against novel, polymorphic, or low-and-slow attacks such as insider threats, credential-stuffing campaigns, and advanced persistent threats (APTs). Artificial Intelligence and Machine Learning (AI/ML) offer a complementary approach: models can learn the statistical notion of "normal" behavior and flag deviations, or learn from labeled historical incidents to recognize attack signatures that were never explicitly coded as rules.

### 2.2 Purpose of this Document

This document serves as the model research reference for the engineering, data science, and SOC teams building the AI-Assisted Threat Detection Dashboard. It documents the problem framing, candidate model families, comparative trade-offs, chosen architecture, feature strategy, evaluation plan, and dashboard requirements needed to move from research to a production-grade system.

### 2.3 Scope

- Network-layer anomaly detection (NetFlow / packet metadata, DNS, proxy logs).
- Endpoint and host-based threat detection (process trees, authentication events).
- User and Entity Behavior Analytics (UEBA) for insider-threat and account-takeover detection.
- A real-time analyst-facing dashboard for alert triage, investigation, and feedback capture.

### 2.4 Objectives

- Achieve high recall on known and novel threat categories while keeping false-positive rate operationally acceptable (target < 2%).
- Provide explainable alerts so analysts can trust and act on model output quickly.
- Support near real-time scoring (sub-second to few-second latency per event/batch).

- Enable continuous learning through analyst feedback (true/false positive labeling) to reduce alert fatigue over time.

### 3. Review of Existing Approaches

Threat detection research broadly falls into four families of technique, each with distinct strengths and blind spots relevant to dashboard design.

| Approach                        | Examples                                                                 | Strengths                                                                   | Limitations                                                  |
|---------------------------------|--------------------------------------------------------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------|
| Signature / Rule-based          | Snort/Suricata rules, YARA, SIEM correlation rules                       | Low false-positive rate on known patterns; fully explainable                | Blind to novel/zero-day attacks; high maintenance overhead   |
| Statistical / Unsupervised ML   | Isolation Forest, One-Class SVM, DBSCAN, PCA reconstruction              | No labeled data required; detects unknown anomalies                         | Higher false-positive rate; anomaly ≠ malicious              |
| Supervised ML                   | Random Forest, Gradient Boosting (XGBoost/LightGBM), Logistic Regression | High accuracy on known attack classes; feature-importance explainability    | Requires large labeled datasets; weak on unseen attack types |
| Deep Learning / Sequence Models | LSTM/GRU, Autoencoders, Transformers, Graph Neural Networks              | Captures temporal & relational patterns; strong on sequential attack chains | Compute-intensive; lower interpretability; needs more data   |

Table 1: Comparative overview of threat detection technique families.

Industry practice (e.g., MITRE ATT&CK-aligned detection engineering and published SOC vendor whitepapers) increasingly favors **hybrid pipelines**: unsupervised models surface "unknown unknowns," supervised classifiers handle high-confidence known-attack scoring, and sequence models detect multi-stage attack chains that unfold over time. This research adopts the same hybrid philosophy.

### 4. Proposed System Architecture

The system is organized into five logical layers, designed to be modular so individual models can be retrained or replaced without disrupting the overall pipeline.

| Layer            | Function                                                                                        | Representative Technologies                                  |
|------------------|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| 1. Ingestion     | Collects logs, flows, and events from firewalls, endpoints, IAM, and cloud audit trails.        | Kafka / Fluentd / Filebeat, Syslog, NetFlow/IPFIX collectors |
| 2. Preprocessing | Parsing, normalization, deduplication, enrichment (GeoIP, asset inventory, threat intel feeds). | Apache Spark / Flink streaming jobs, Python ETL              |
| 3. Feature Store | Computes and serves rolling statistical, behavioral, and graph features per entity.             | Feast / Redis / time-series feature tables                   |

| Layer                   | Function                                                                                | Representative Technologies                                    |
|-------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------|
| 4. Model Layer          | Ensemble of unsupervised, supervised, and sequence models producing a fused risk score. | Scikit-learn, XGBoost, PyTorch/TensorFlow, MLflow for registry |
| 5. Dashboard & Response | Real-time alert queue, investigation views, feedback capture, and case management.      | React front end, WebSocket/GraphQL API, SOAR integration       |

Table 2: End-to-end architecture layers.

4.1 Data Flow Summary

Raw events stream through ingestion and preprocessing, are converted into entity-centric feature vectors (per user, host, or IP), and are scored in parallel by the model ensemble. A fusion layer combines individual model scores into a single calibrated risk score (0–100) with contributing-reason tags, which is what analysts see first on the dashboard.

5. Data Collection and Preprocessing

5.1 Data Sources

- **Network telemetry:** NetFlow/IPFIX records, DNS query logs, proxy/firewall logs, TLS handshake metadata (JA3/JA3S fingerprints).
- **Endpoint telemetry:** Process creation events, command-line arguments, file and registry access, EDR alerts.
- **Identity & access:** Authentication logs (success/failure, geo, device), privilege escalation events, VPN sessions.
- **Threat intelligence:** IOC feeds (malicious IPs/domains/hashes), MITRE ATT&CK technique mappings.
- **Public benchmark datasets (for offline model validation):** CICIDS2017/2018, UNSW-NB15, NSL-KDD, and CERT Insider Threat dataset.

5.2 Preprocessing Pipeline

- **Parsing & normalization:** Convert heterogeneous log formats into a common schema (e.g., ECS — Elastic Common Schema).
- **Deduplication & sessionization:** Group related events into sessions/flows to reduce noise.
- **Enrichment:** Attach GeoIP, asset criticality, user role, and threat-intel reputation scores.
- **Handling class imbalance:** Malicious events are typically <1% of traffic; techniques such as SMOTE, class-weighting, and focal loss are used for supervised models.
- **Missing data:** Forward-fill for time-series gaps; sentinel encoding for categorical nulls.

5.3 Data Governance and Privacy

Personally identifiable information (usernames, IPs tied to individuals) is pseudonymized in the feature store, with re-identification only permitted through an access-controlled lookup used during confirmed-incident investigation, in line with organizational data-protection policy.

## 6. Model Research and Methodology

---

Three model families were evaluated in parallel during research, reflecting the hybrid architecture described in Section 4.

### 6.1 Unsupervised Anomaly Detection

Used to catch novel or zero-day behavior that has no labeled precedent. Candidate models evaluated:

- **Isolation Forest** — efficient, scales well to high-dimensional tabular features; used as the primary baseline for host/network anomaly scoring.
- **Autoencoder (deep, dense/convolutional)** — learns a compressed representation of "normal" traffic; reconstruction error above a threshold flags anomalies. Effective on high-dimensional NetFlow features.
- **One-Class SVM** — evaluated for smaller, well-bounded feature sets (e.g., per-user login behavior); found to scale poorly beyond ~100K samples.
- **DBSCAN / HDBSCAN clustering** — used offline for threat-hunting and cluster discovery rather than real-time scoring, due to computational cost.

### 6.2 Supervised Classification

Used where labeled historical incident data exists (confirmed malware, phishing, brute-force, SQL-injection, C2 beaconing, etc.).

- **Random Forest** — strong baseline; robust to noisy features and provides feature-importance for analyst explainability.
- **Gradient Boosted Trees (XGBoost / LightGBM)** — best raw accuracy in benchmark testing on CICIDS2018-style features; selected as the primary supervised classifier.
- **Logistic Regression** — retained as an interpretable, low-latency fallback model for lightweight edge deployments.

### 6.3 Deep Learning / Sequential Models

Used to capture multi-step attack chains and temporal dependencies that point-in-time models miss.

- **LSTM / GRU networks** — model sequences of authentication and process events per entity to detect slow, staged attacks (e.g., lateral movement).
- **Transformer-based sequence encoders** — evaluated for longer-context session modeling; higher accuracy but materially higher training/inference cost.
- **Graph Neural Networks (GNNs)** — modeled the host-to-host and user-to-resource access graph to detect anomalous lateral-movement edges; promising in offline evaluation, planned for a later research phase.

### 6.4 Model Comparison Summary

| Model                | Paradigm      | Relative Accuracy*      | Interpretability | Inference Latency |
|----------------------|---------------|-------------------------|------------------|-------------------|
| Isolation Forest     | Unsupervised  | Medium                  | Medium           | Low               |
| Autoencoder          | Unsupervised  | Medium–High             | Low              | Low–Medium        |
| Random Forest        | Supervised    | High                    | High             | Low               |
| XGBoost / LightGBM   | Supervised    | Highest                 | Medium–High      | Low               |
| LSTM / GRU           | Sequential DL | High (temporal attacks) | Low              | Medium            |
| Graph Neural Network | Relational DL | High (lateral movement) | Low              | Medium–High       |

Table 3: Relative comparison based on internal benchmark testing on CICIDS-style datasets. \*Accuracy figures are directional, not absolute, and depend on feature quality and tuning.

### 6.5 Selected Hybrid Ensemble

Based on the comparison above, the recommended production model is a **three-stage hybrid ensemble**:

- **Stage 1 — Fast filter:** Isolation Forest / Autoencoder anomaly score computed on every event at low cost to discard clearly benign traffic.
- **Stage 2 — Classification:** Events surviving Stage 1 are scored by the XGBoost classifier against known attack categories, producing a calibrated probability and MITRE ATT&CK technique tag.
- **Stage 3 — Sequence context:** An LSTM-based session model re-scores the entity's recent event window to catch multi-stage attacks that any single event would miss.
- A weighted fusion function combines the three scores into the final Risk Score (0–100) shown on the dashboard, with SHAP-based feature attribution attached for analyst explainability.

## 7. Feature Engineering

Feature quality is the largest single driver of detection performance observed in this research. Features are grouped into four categories:

| Category                | Examples                                                                                            |
|-------------------------|-----------------------------------------------------------------------------------------------------|
| Volumetric              | Bytes/packets per flow, connection count per minute, unique destination ports contacted             |
| Behavioral / Temporal   | Login time-of-day deviation, rolling average session duration, time since last privileged action    |
| Relational / Graph      | Degree centrality of a host in the access graph, number of distinct hosts an account touched in 24h |
| Contextual / Enrichment | Threat-intel reputation score, asset criticality tier, geo-velocity (impossible travel)             |

Table 4: Feature categories used across the model ensemble.

All features are computed as rolling windows (5-minute, 1-hour, 24-hour) per entity to capture both burst behavior and slow drift, and are versioned in the feature store so that training and serving pipelines remain

consistent (avoiding training/serving skew).

## 8. Evaluation Methodology

### 8.1 Metrics

Because malicious events are rare relative to benign traffic, accuracy alone is misleading. The following metrics are used:

| Metric                     | Why it matters                                       | Target      |
|----------------------------|------------------------------------------------------|-------------|
| Precision                  | Minimizes analyst time wasted on false alerts        | > 90%       |
| Recall (Detection Rate)    | Minimizes missed real attacks                        | > 95%       |
| F1-Score                   | Balances precision and recall for imbalanced classes | > 0.92      |
| ROC-AUC / PR-AUC           | Threshold-independent model quality comparison       | > 0.95 AUC  |
| False Positive Rate        | Directly drives analyst trust and alert fatigue      | < 2%        |
| Mean Time to Detect (MTTD) | Operational KPI beyond model metrics                 | < 5 minutes |

Table 5: Evaluation metrics and target thresholds for production readiness.

### 8.2 Validation Strategy

- **Time-based split** (not random split) to avoid data leakage, since attack patterns evolve over time.
- **Red-team / purple-team exercises** to test detection against live simulated attacks (e.g., using MITRE ATT&CK emulation plans / Atomic Red Team).
- **Shadow deployment** — new models run alongside production models for 2–4 weeks, scored but not alerting, before promotion.
- **Analyst feedback loop** — every alert resolution (true positive / false positive / benign) is captured and fed back into periodic retraining.

## 9. Dashboard Design and User Experience

### 9.1 Design Principles

- **Risk-first triage:** Alerts are ranked by fused Risk Score, not just raw event volume, so analysts see the highest-priority items first.
- **Explainability at a glance:** Each alert card shows the top contributing features (via SHAP values) and the matched MITRE ATT&CK technique.
- **Progressive disclosure:** Summary view for triage, drill-down view for full event timeline and raw log inspection.
- **One-click feedback:** Analysts confirm/dismiss alerts inline, directly feeding the retraining pipeline.



9.2 Core Dashboard Views

| View               | Purpose                                                                                     |
|--------------------|---------------------------------------------------------------------------------------------|
| Live Threat Feed   | Real-time, risk-ranked stream of alerts with entity, technique tag, and confidence.         |
| Entity Timeline    | Chronological view of all events for a given user/host, highlighting anomalous points.      |
| Network Graph View | Interactive graph of host/user relationships to visualize lateral movement.                 |
| Model Health Panel | Tracks precision/recall drift, feature drift, and retraining status for MLOps oversight.    |
| Executive Summary  | Daily/weekly rollup of threat trends, top techniques, and SOC response KPIs for leadership. |

Table 6: Primary dashboard views and their purpose.

10. Proposed Technology Stack

| Layer                 | Suggested Tools                                                                |
|-----------------------|--------------------------------------------------------------------------------|
| Streaming / Messaging | Apache Kafka, AWS Kinesis                                                      |
| Stream Processing     | Apache Flink / Spark Structured Streaming                                      |
| Feature Store         | Feast, Redis, TimescaleDB                                                      |
| Model Training        | Scikit-learn, XGBoost/LightGBM, PyTorch                                        |
| Model Serving         | MLflow, Seldon/BentoML, ONNX Runtime for low-latency inference                 |
| Storage               | Elasticsearch/OpenSearch (logs), S3/Parquet (cold data), PostgreSQL (metadata) |
| Dashboard Frontend    | React + WebSockets, D3.js/Recharts for visualizations                          |
| Orchestration         | Kubernetes, Airflow for scheduled retraining jobs                              |

Table 7: Suggested technology stack per architectural layer.

11. Implementation Roadmap

| Phase                                  | Duration  | Key Deliverables                                                                        |
|----------------------------------------|-----------|-----------------------------------------------------------------------------------------|
| Phase 1 — Research & Data Readiness    | 4–6 weeks | Finalize data sources, build ingestion pipeline, establish feature store schema.        |
| Phase 2 — Baseline Models              | 4 weeks   | Train Isolation Forest and Random Forest/XGBoost baselines; offline evaluation report.  |
| Phase 3 — Sequential & Ensemble Models | 6 weeks   | Train LSTM session model; build fusion layer; SHAP explainability integration.          |
| Phase 4 — Dashboard MVP                | 6 weeks   | Live Threat Feed, Entity Timeline, and feedback capture UI; integration with model API. |
| Phase 5 — Shadow Deployment & Tuning   | 3–4 weeks | Run in parallel with existing SIEM alerts; calibrate thresholds; red-team validation.   |

| Phase                        | Duration | Key Deliverables                                                               |
|------------------------------|----------|--------------------------------------------------------------------------------|
| Phase 6 — Production Rollout | Ongoing  | Phased rollout to SOC teams; continuous retraining pipeline; MLOps monitoring. |

Table 8: Proposed phased implementation roadmap.

## 12. Challenges and Limitations

- **Concept drift:** Attacker behavior and normal business traffic both evolve, requiring scheduled retraining and drift monitoring.
- **Class imbalance:** Malicious events remain a very small fraction of total traffic, complicating supervised training and evaluation.
- **Adversarial evasion:** Sophisticated attackers may attempt to blend in with normal traffic patterns or poison training data; adversarial robustness testing is required.
- **Alert fatigue:** Even a low false-positive rate can generate a large absolute number of alerts at enterprise scale; the risk-fusion and prioritization design directly targets this.
- **Explainability vs. accuracy trade-off:** The most accurate models (deep sequence models, GNNs) are also the least interpretable, requiring investment in SHAP/LIME-based explanation tooling.
- **Data privacy and compliance:** Behavioral monitoring of users must align with organizational policy and applicable data-protection regulations.

## 13. Future Research Directions

- **Graph Neural Networks** for full-scale lateral-movement and insider-threat detection across the enterprise access graph.
- **Federated learning** to train shared detection models across business units or partner organizations without centralizing raw log data.
- **LLM-assisted alert summarization** to auto-generate natural-language incident summaries and suggested response playbooks for analysts.
- **Active learning** to prioritize which uncertain alerts are sent to analysts for labeling, maximizing the value of every feedback label.
- **Self-supervised pretraining** on large volumes of unlabeled log data to reduce dependency on scarce labeled attack examples.

## 14. Conclusion

This research establishes that no single model family is sufficient for comprehensive threat detection: unsupervised models are needed for novel threats, supervised classifiers for known-attack precision, and sequential/graph models for multi-stage attack chains. The proposed hybrid ensemble — combined with a risk-first, explainable dashboard and a continuous analyst-feedback loop — provides a practical path to a production-ready AI-Assisted Threat Detection Dashboard that meaningfully reduces Mean Time to Detect while keeping the false-positive burden on SOC analysts manageable.

## 15. References and Further Reading

---

- MITRE ATT&CK Framework — [attack.mitre.org](https://attack.mitre.org)
- Sharafaldin, I. et al., "Toward Generating a New Intrusion Detection Dataset (CICIDS2017/2018)," Canadian Institute for Cybersecurity.
- Moustafa, N. & Slay, J., "UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems."
- CERT Insider Threat Center Dataset, Carnegie Mellon University Software Engineering Institute.
- Chandola, V., Banerjee, A., Kumar, V., "Anomaly Detection: A Survey," ACM Computing Surveys.
- Atomic Red Team — attack emulation library for detection validation ([redcanary.co](https://redcanary.co)).
- Lundberg, S. & Lee, S., "A Unified Approach to Interpreting Model Predictions (SHAP)," NeurIPS.